

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5
Специальность ПО-13

Выполнил:
студент 3 курса
группы ПО-13
А.В. Король
Проверил:
А.Д.Кулик
01.04.2026

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Задание 1.

1. Реализовать базу данных из не менее **5 таблиц** на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
 2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
 3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
 4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
 5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпоинт;
- 9) База данных Справочное бюро ж/д вокзала

Выполнение:

Код программы:

```
from datetime import datetime

from fastapi import FastAPI
from sqlalchemy import Column
from sqlalchemy import DateTime
from sqlalchemy import Float
from sqlalchemy import ForeignKey
from sqlalchemy import Integer
from sqlalchemy import String
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base
from sqlalchemy.orm import relationship
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "sqlite:///railway_station.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False},
)

SessionLocal = sessionmaker(bind=engine)

Base = declarative_base()

app = FastAPI()

class Station(Base):
    __tablename__ = "stations"

    id = Column(Integer, primary_key=True)
```

```

name = Column(String, nullable=False)
city = Column(String, nullable=False)

class Train(Base):
    __tablename__ = "trains"

    id = Column(Integer, primary_key=True)
    number = Column(String, nullable=False)
    departure_station = Column(String, nullable=False)
    arrival_station = Column(String, nullable=False)
    price = Column(Float, nullable=False)

    schedules = relationship("Schedule", back_populates="train")
    tickets = relationship("Ticket", back_populates="train")

class Passenger(Base):
    __tablename__ = "passengers"

    id = Column(Integer, primary_key=True)
    full_name = Column(String, nullable=False)
    passport = Column(String, nullable=False)

    tickets = relationship("Ticket", back_populates="passenger")

class Schedule(Base):
    __tablename__ = "schedules"

    id = Column(Integer, primary_key=True)

    train_id = Column(Integer, ForeignKey("trains.id"))

    departure_time = Column(String, nullable=False)
    arrival_time = Column(String, nullable=False)
    trip_date = Column(String, nullable=False)

    train = relationship("Train", back_populates="schedules")

class Ticket(Base):
    __tablename__ = "tickets"

    id = Column(Integer, primary_key=True)

    passenger_id = Column(Integer, ForeignKey("passengers.id"))

    train_id = Column(Integer, ForeignKey("trains.id"))

    seat_number = Column(String, nullable=False)

    purchase_date = Column(DateTime, default=datetime.utcnow)

```

```
passenger = relationship("Passenger", back_populates="tickets")
train = relationship("Train", back_populates="tickets")
```

```
Base.metadata.create_all(bind=engine)
```

```
@app.get("/")
def home():
    return {"message": "Railway Station API"}
```

```
@app.post("/stations")
def add_station(name: str, city: str):
    session = SessionLocal()

    station = Station(
        name=name,
        city=city,
    )

    session.add(station)

    session.commit()

    session.close()

    return {"message": "Station added"}
```

```
@app.get("/stations")
def get_stations():
    session = SessionLocal()

    stations = session.query(Station).all()

    result = []

    for station in stations:
        result.append(
            {
                "id": station.id,
                "name": station.name,
                "city": station.city,
            }
        )

    session.close()

    return result
```

```
@app.post("/trains")
def add_train(
```

```

        number: str,
        departure_station: str,
        arrival_station: str,
        price: float,
    ):
        session = SessionLocal()

        train = Train(
            number=number,
            departure_station=departure_station,
            arrival_station=arrival_station,
            price=price,
        )

        session.add(train)

        session.commit()

        session.close()

        return {"message": "Train added"}

@app.get("/trains")
def get_trains():
    session = SessionLocal()

    trains = session.query(Train).all()

    result = []

    for train in trains:
        result.append(
            {
                "id": train.id,
                "number": train.number,
                "departure_station": train.departure_station,
                "arrival_station": train.arrival_station,
                "price": train.price,
            }
        )

    session.close()

    return result

@app.put("/trains/{train_id}")
def update_train(
    train_id: int,
    number: str,
    departure_station: str,
    arrival_station: str,
    price: float,

```

```

):
    session = SessionLocal()

    train = session.query(Train).filter(Train.id == train_id).first()

    if not train:
        session.close()

        return {"message": "Train not found"}

    train.number = number
    train.departure_station = departure_station
    train.arrival_station = arrival_station
    train.price = price

    session.commit()

    session.close()

    return {"message": "Train updated"}

@app.delete("/trains/{train_id}")
def delete_train(train_id: int):
    session = SessionLocal()

    train = session.query(Train).filter(Train.id == train_id).first()

    if not train:
        session.close()

        return {"message": "Train not found"}

    session.delete(train)

    session.commit()

    session.close()

    return {"message": "Train deleted"}

@app.post("/passengers")
def add_passenger(full_name: str, passport: str):
    session = SessionLocal()

    passenger = Passenger(
        full_name=full_name,
        passport=passport,
    )

    session.add(passenger)

    session.commit()

```

```

    session.close()

    return {"message": "Passenger added"}

@app.get("/passengers")
def get_passengers():
    session = SessionLocal()

    passengers = session.query(Passenger).all()

    result = []

    for passenger in passengers:
        result.append(
            {
                "id": passenger.id,
                "full_name": passenger.full_name,
                "passport": passenger.passport,
            }
        )

    session.close()

    return result

@app.post("/schedules")
def add_schedule(
    train_id: int,
    departure_time: str,
    arrival_time: str,
    trip_date: str,
):
    session = SessionLocal()

    schedule = Schedule(
        train_id=train_id,
        departure_time=departure_time,
        arrival_time=arrival_time,
        trip_date=trip_date,
    )

    session.add(schedule)

    session.commit()

    session.close()

    return {"message": "Schedule added"}

@app.get("/schedules")

```

```

def get_schedules():
    session = SessionLocal()

    schedules = session.query(Schedule).all()

    result = []

    for schedule in schedules:
        result.append(
            {
                "id": schedule.id,
                "train_id": schedule.train_id,
                "departure_time": schedule.departure_time,
                "arrival_time": schedule.arrival_time,
                "trip_date": schedule.trip_date,
            }
        )

    session.close()

    return result

```

```

@app.post("/tickets")
def buy_ticket(
    passenger_id: int,
    train_id: int,
    seat_number: str,
):
    session = SessionLocal()

    ticket = Ticket(
        passenger_id=passenger_id,
        train_id=train_id,
        seat_number=seat_number,
    )

    session.add(ticket)

    session.commit()

    session.close()

    return {"message": "Ticket purchased"}

```

```

@app.get("/tickets")
def get_tickets():
    session = SessionLocal()

    tickets = session.query(Ticket).all()

    result = []

```

```

for ticket in tickets:
    result.append(
        {
            "id": ticket.id,
            "passenger_id": ticket.passenger_id,
            "train_id": ticket.train_id,
            "seat_number": ticket.seat_number,
            "purchase_date": ticket.purchase_date,
        }
    )

session.close()

return result

```

Рисунки с результатами работы программы

FastAPI 0.1.0 OAS 3.1
[/openapi.json](#)

default

GET	/ Home	⌵
POST	/stations Add Station	⌵
GET	/stations Get Stations	⌵
POST	/trains Add Train	⌵
GET	/trains Get Trains	⌵
PUT	/trains/{train_id} Update Train	⌵
DELETE	/trains/{train_id} Delete Train	⌵
POST	/passengers Add Passenger	⌵
GET	/passengers Get Passengers	⌵
POST	/schedules Add Schedule	⌵
GET	/schedules Get Schedules	⌵
POST	/tickets Buy Ticket	⌵
GET	/tickets Get Tickets	⌵

```
PS C:\Users\user\Documents\GitHub\spp_po13\reports\Koro1\lab5\src> python -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\user\\Documents\\GitHub\\spp_po13\\reports\\Koro1\\lab5\\src']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [25152] using StatReload
INFO: Started server process [23948]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:56975 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:56975 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:57573 - "POST /stations?name=Moscow%20Station&city=Moscow HTTP/1.1" 200 OK
INFO: 127.0.0.1:58703 - "POST /trains?number=A101&departure_station=Moscow&arrival_station=SPB&price=1500 HTTP/1.1" 200 OK
INFO: 127.0.0.1:53307 - "POST /passengers?full_name=Ivan%20Ivanov&passport=123456 HTTP/1.1" 200 OK
INFO: 127.0.0.1:57821 - "POST /schedules?train_id=1&departure_time=10%3A00&arrival_time=14%3A00&trip_date=2026-05-15 HTTP/1.1" 200 OK
INFO: 127.0.0.1:49797 - "POST /tickets?passenger_id=1&train_id=1&seat_number=12A HTTP/1.1" 200 OK
INFO: 127.0.0.1:65316 - "GET /trains HTTP/1.1" 200 OK
INFO: 127.0.0.1:50837 - "GET /stations HTTP/1.1" 200 OK
INFO: 127.0.0.1:53327 - "GET /passengers HTTP/1.1" 200 OK
INFO: 127.0.0.1:50257 - "GET /schedules HTTP/1.1" 200 OK
INFO: 127.0.0.1:54199 - "GET /tickets HTTP/1.1" 200 OK
INFO: 127.0.0.1:53083 - "GET / HTTP/1.1" 200 OK
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [23948]
INFO: Stopping reloader process [25152]
```

Вывод: приобрёл практические навыки разработки API и баз данных